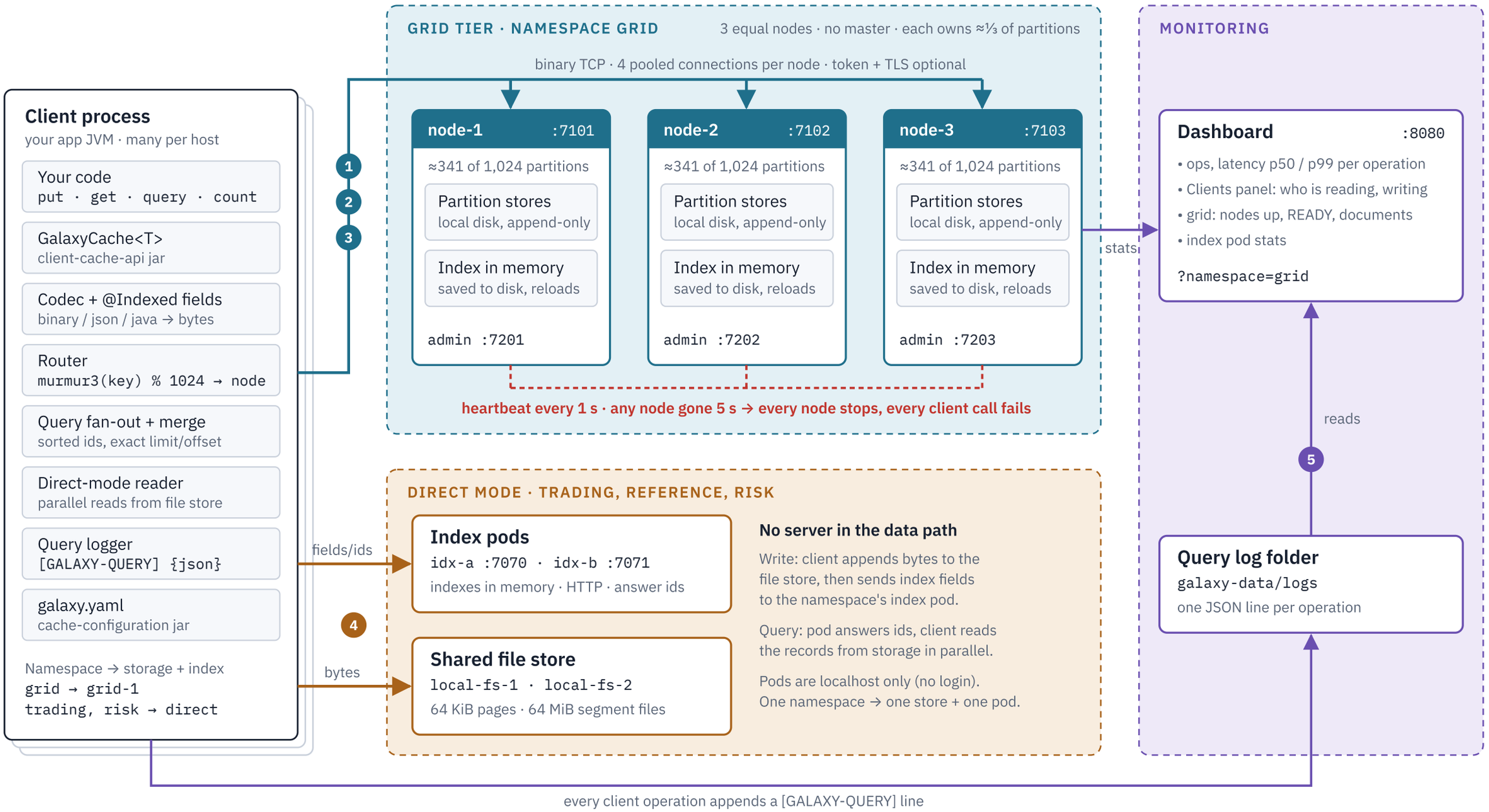
Galaxy Cache: how it fits together

A Java key-value cache. Apps use one client jar; data lives either on a 3-node grid (data and index on each node) or in direct mode (shared file store plus index pods). Every operation is logged for the dashboard.

`Java 21` `1,024 partitions` `binary TCP` `fail-stop · no backups`



1 Write to the grid

Client encodes each object and pulls its `@Indexed` fields, hashes the key to a partition, and sends one batch per owner node in parallel. The node appends to disk, fsyncs, then updates its index.

2 Read by key

`get` / `getAll` go only to the owner node: one round trip, no index call. The node streams stored bytes back; the client decodes them on its read threads.

3 Query and count

Sent to all 3 nodes at once. Each searches its own index (a third of the data), returns sorted ids or records; the client merges, so limit and offset are exact. Counts are summed.

4 Direct mode

Same API, chosen in `galaxy.yaml`. Client writes bytes to the shared file store and fields to the index pod; queries get ids from the pod, then read storage in parallel.

5 Monitoring

Every client operation writes a query-log line. The dashboard reads that folder plus each node's admin port (health, stats, Prometheus `/metrics`).

Failure rule: fail-stop. Recovery is start all nodes, then restart clients. **One copy** of each partition; data is safe on the node's disk.

Not built yet: backups and failover (Phase 2), coordinator / thin-client queries, live rebalancing.